```
> ##################################################################
  #  gamma_wave_maple.mpl
  #  TauTimp — Gamma-Ray Longitudinal Wave Equation
  #  Full Derivation, Solution, Casimir Pressure, and 12 Plots
  #  R  = 3.9228e22 m   (electron-scale boundary parameter)
  #  Rs = 7.7665e25 m   (cosmological boundary parameter)
  #  V  = 1e9 * c       (superluminal phase velocity)
  #  Author : Viorel Popescu — TauTimp Project 2026
  #  Execute: !!! (Execute All) in Maple 2022+
  ##################################################################

  restart;
  with(plots): with(plottools): with(LinearAlgebra):

  ##################################################################
  # SECTION 0 — Physical constants and key numerics
  ##################################################################
  printf("=== SECTION 0 : Physical constants ===\n");

  c    := 2.99792458e8:           # speed of light         [m/s]
  hbar := 1.054571817e-34:        # reduced Planck constant [J•s]
  Pi_  := evalf(Pi):              # numerical Pi

  R    := 3.9228e22:              # electron boundary param [m]
  Rs   := 7.7665e25:              # cosmological boundary   [m]
  V    := 1e9 * c:                # superluminal speed      [m/s]

  # Derived
  V2   := evalf(V^2):
  RsR  := evalf(Rs + R):
  L    := evalf(2*(Rs - R)):      # Casimir plate separation [m]

  # Airy zeros  a_n  (zeros of Ai(-zeta), i.e. positive zeros of Ai with
  negated arg)
  a1 := 2.338107411:
  a2 := 4.087949444:
  a3 := 5.520559828:

  # Eigenvalues  lambda_n = a_n^{3/2} / (2^{3/2} * R)
  lam1 := evalf(a1^(3/2) / (2^(3/2) * R)):
  lam2 := evalf(a2^(3/2) / (2^(3/2) * R)):
  lam3 := evalf(a3^(3/2) / (2^(3/2) * R)):

  # Angular frequencies  omega_n = lambda_n * V
  om1  := evalf(lam1 * V):
  om2  := evalf(lam2 * V):
  om3  := evalf(lam3 * V):
```

```
# Polynomial coefficient c2
c2   := evalf(-R * V2 / RsR):

# Numerator polynomial coefficients  N(r) = alpha0 + alpha1*r + alpha2*r^2
alpha0 := evalf(-4*R^2 * Rs / RsR):
alpha1 := evalf(2*R):
alpha2 := evalf(-R / RsR):

printf("  R       = %g m\n",     R);
printf("  Rs      = %g m\n",     Rs);
printf("  V       = %g m/s\n",   V);
printf("  V^2     = %g m^2/s^2\n", V2);
printf("  Rs+R    = %g m\n",     RsR);
printf("  L=2(Rs-R) = %g m\n",   L);
printf("  a1      = %g\n",       a1);
printf("  a2      = %g\n",       a2);
printf("  a3      = %g\n",       a3);
printf("  lambda1 = %g m^-1\n",  lam1);
printf("  lambda2 = %g m^-1\n",  lam2);
printf("  lambda3 = %g m^-1\n",  lam3);
printf("  omega1  = %g rad/s\n", om1);
printf("  omega2  = %g rad/s\n", om2);
printf("  omega3  = %g rad/s\n", om3);
printf("  c2      = %g m^2/s^2\n", c2);
printf("  alpha0  = %g m^2\n",   alpha0);
printf("  alpha1  = %g m\n",     alpha1);
printf("  alpha2  = %g  (dim-less)\n", alpha2);

################################################################
# SECTION 1 — Symbolic PDE in spherical symmetry
################################################################
printf("\n=== SECTION 1 : Symbolic PDE (spherical) ===\n");

# Field  H(r,theta,t),  spherically symmetric part  f(r,t)
# PDE:  (R/r)*[ f_rr + (2/r)*f_r ] = (1/V^2)*f_tt

f_sym := f(r, t):

PDE_full := (R/r) * ( diff(f_sym, r, r)
                    + (2/r)*diff(f_sym, r) )
          - (1/V^2) * diff(f_sym, t, t) = 0;

printf("  PDE_full := ");
print(PDE_full);
printf("  (spherically symmetric, azimuthal symmetry)\n");

################################################################
# SECTION 2 — Separation of variables  f = g(r)*u(t)
################################################################
```

```
printf("\n=== SECTION 2 : Separation of variables ===\n");

# Ansatz f = g(r)*u(t) => two ODEs with separation constant -lambda^2:
#
#   (T)  u''  + lambda^2 * V^2 * u = 0
#   (R)  g''  + (2/r)*g' + (lambda^2*r/R)*g = 0

printf("  ODE (T): u'' + lambda^2*V^2*u = 0\n");
printf("  ODE (R): g'' + (2/r)*g' + (lambda^2*r/R)*g = 0\n\n");

# Solve temporal ODE symbolically
ODE_T := diff(u(t),t,t) + lam1^2 * V2 * u(t) = 0;
sol_T := dsolve(ODE_T, u(t));
printf("  u(t) [n=1] = "); print(sol_T);

# Radial ODE: substitution g(r) = v(r)/r => v'' + (lambda^2*r/R)*v = 0
(Airy)
# xi = -(lambda^2/R)^{1/3} * r  =>  d^2v/dxi^2 - xi*v = 0
printf("  Substitution g=v/r => Airy eq:  v'' + (lambda^2*r/R)*v = 0\n");
printf("  xi = -(lambda^2/R)^{1/3}*r  =>  d^2v/dxi^2 - xi*v = 0\n");
printf("  v = C1*AiryAi(xi) + C2*AiryBi(xi)\n");
printf("  g(r) = v(r)/r\n");

################################################################
# SECTION 3 — Airy solution and regularity at r=0
################################################################
printf("\n=== SECTION 3 : Airy solution, regularity at r=0 ===\n");

# Ai(0) and Bi(0) values
Ai0 := evalf(AiryAi(0)):
Bi0 := evalf(AiryBi(0)):
printf("  Ai(0) = %g  (= 3^{-2/3}/Gamma(2/3))\n", Ai0);
printf("  Bi(0) = %g\n", Bi0);
printf("  Regularity: v(0)=0 requires C1*Ai(0)+C2*Bi(0)=0\n");
printf("  Physical branch: C2=0 (dominant Ai), C1 != 0\n");
printf("  => g(r) = (C1/r)*AiryAi(-(lambda^2/R)^{1/3}*r)\n");

# Radial eigenfunction for n=1 (illustrative proc)
xi_n := proc(rr, lam)  -(lam^2/R)^(1/3) * rr  end proc:
g_n  := proc(rr, lam)
    if rr < 1e10 then 1e10   # avoid /0
    else AiryAi(xi_n(rr, lam)) / rr
    end if
end proc:

# Safe proc for plotting (avoid r=0 singularity)
g1_plot := proc(rr)  AiryAi(xi_n(rr, lam1)) / rr  end proc:
g2_plot := proc(rr)  AiryAi(xi_n(rr, lam2)) / rr  end proc:
g3_plot := proc(rr)  AiryAi(xi_n(rr, lam3)) / rr  end proc:
```

```
##############################################################
# SECTION 4 — Eigenvalue quantisation  BC: g(2R)=0
##############################################################
printf("\n=== SECTION 4 : Eigenvalue quantisation  g(2R)=0 ===\n");

# Condition: AiryAi(-(lambda_n^2/R)^{1/3}*2R) = 0
# => (lambda_n^2/R)^{1/3}*2R = a_n  (n-th zero of Ai(-zeta))
# => lambda_n = a_n^{3/2} / (2^{3/2} * R)

for n from 1 to 3 do
    an_  := [a1,a2,a3][n]:
    lamn := an_^(3/2) / (2^(3/2)*R):
    arg  := evalf(-(lamn^2/R)^(1/3) * 2*R):
    val  := evalf(AiryAi(arg)):
    printf("  n=%d  a_%d=%.6f  lambda_%d=%g m^-1  Ai(arg)=%g (->0)\n",
           n, n, an_, n, lamn, val);
end do;

##############################################################
# SECTION 5 — Full Airy-mode expansion  f(r,t)
##############################################################
printf("\n=== SECTION 5 : Full field f(r,t) — Airy mode expansion ===\n")
;

# f(r,t) = sum_n (1/r)*Ai(-(lambda_n^2/R)^{1/3}*r)
#                      *[A_n*cos(omega_n*t)+B_n*sin(omega_n*t)]

printf("  f(r,t) = SUM_n (1/r)*Ai(-(lam_n^2/R)^{1/3}*r)\n");
printf("                   *[A_n*cos(om_n*t)+B_n*sin(om_n*t)]\n\n");

# Numerical spot-checks for n=1 mode (A1=1, B1=0) at t=0
f1_t0 := proc(rr)  AiryAi(xi_n(rr, lam1)) / rr  end proc:

printf("  f_1(R,  0)  = %g\n",   evalf(f1_t0(R)));
printf("  f_1(2R, 0)  = %g  (BC => ~0)\n", evalf(f1_t0(2*R)));
printf("  f_1(0.5R,0) = %g\n",   evalf(f1_t0(0.5*R)));

##############################################################
# SECTION 6 — Casimir reduced equation and polynomial solution
##############################################################
printf("\n=== SECTION 6 : Casimir reduced PDE (1+1) and polynomial f(r) ==
=\n");

# 1+1 reduced Casimir PDE:  (1/V^2)*f_tt - f_rr = 0
# Static particular solution f(r):
#   f(r) = [alpha0 + alpha1*r + alpha2*r^2] / (8*Pi*r^2)

f_static := proc(rr)
    (alpha0 + alpha1*rr + alpha2*rr^2) / (8*Pi_*rr^2)
```

```
end proc:

printf("  f(r) = [alpha0 + alpha1*r + alpha2*r^2] / (8*Pi*r^2)\n");
printf("  alpha0 = %g m^2\n",    alpha0);
printf("  alpha1 = %g m\n",      alpha1);
printf("  alpha2 = %g\n",        alpha2);
printf("  c2     = %g m^2/s^2\n", c2);

# BC verification
f_2R  := evalf(f_static(2*R)):
f_2Rs := evalf(f_static(2*Rs)):
printf("  BC check  f(2R)  = %g  (should be ~0)\n",  f_2R);
printf("  BC check  f(2Rs) = %g  (should be ~0)\n", f_2Rs);

##############################################################
# SECTION 7 — Explicit boxed form of f(r) after c2 substitution
##############################################################
printf("\n=== SECTION 7 : Explicit f(r) with c2 substituted ===\n");

# f(r) = (1/(8*Pi)) * [ -4*R^2*Rs/((Rs+R)*r^2) + 2*R/r - R/(Rs+R) ]

f_box := proc(rr)
    (1/(8*Pi_)) * ( -4*R^2*Rs/((Rs+R)*rr^2)
                    + 2*R/rr
                    - R/(Rs+R) )
end proc:

printf("  f(r) = (1/8Pi)*[-4R^2*Rs/((Rs+R)*r^2) + 2R/r - R/(Rs+R)]\n\n");

# Verify equivalence with polynomial form
diff_check := evalf(f_box(3*R) - f_static(3*R)):
printf("  Equivalence check at r=3R:  f_box - f_poly = %g\n", diff_check);

##############################################################
# SECTION 8 — Particle flux Phi(r) and Casimir pressure
##############################################################
printf("\n=== SECTION 8 : Particle flux Phi(r) and Casimir pressure ===
\n");

# Flux:  Phi(r) = f(r) / (4*Pi*r^2)
#              = [alpha0 + alpha1*r + alpha2*r^2] / (32*Pi^2*r^4)
# Falls as r^{-4} for large r

Phi_r := proc(rr)
    f_static(rr) / (4*Pi_*rr^2)
end proc:

printf("  Phi(r) = f(r)/(4*Pi*r^2)  [~r^-4 at large r]\n");
printf("  Phi(R)    = %g\n",   evalf(Phi_r(R)));
```

```
printf("  Phi(2R)   = %g\n",   evalf(Phi_r(2*R)));
printf("  Phi(10R)  = %g\n",   evalf(Phi_r(10*R)));
printf("  Phi(100R) = %g\n",   evalf(Phi_r(100*R)));

# Casimir pressure (symbolic form)
printf("\n  Casimir pressure:\n");
printf("  P_Casimir = hbar*V / (2*L) * SUM_n omega_n\n");
printf("  F_Casimir ~ hbar*V / L^2  (enhanced by factor V/c = 1e9)\n");
printf("  L = 2(Rs-R) = %g m\n", L);

# Leading term hbar*V/L^2
Pcas := evalf(hbar * V / L^2):
printf("  hbar*V/L^2 = %g  N/m^2  (Casimir scale)\n", Pcas);

################################################################
# SECTION 9 — Legendre reconstruction H(r,theta,t)
################################################################
printf("\n=== SECTION 9 : Full field H(r,theta,t) with Legendre expansion
===\n");

# H(r,theta,t) = SUM_{n,l} (1/r)*Ai(-(lam_{nl}^2/R)^{1/3}*r)
#                           * P_l(cos theta)
#                           * [A_{nl}*cos(om_{nl}*t)+B_{nl}*sin(om_{nl}*t)
]

printf("  H = SUM_{n,l} (1/r)*Ai(-arg)*P_l(cos(theta))*[A cos+B sin]\n\n")
;

# Numerical check of Legendre modes at theta = Pi/4
theta0 := Pi_/4:
for l from 0 to 2 do
    Pl := evalf(LegendreP(l, cos(theta0))):
    printf("  P_%d(cos(Pi/4)) = %g\n", l, Pl);
end do;

# Sample radial profile for (n=1, l=0) at t=0
printf("\n  Mode (n=1,l=0) radial profile at t=0, theta=Pi/4:\n");
printf("  H(R,  Pi/4, 0) = %g\n", evalf(f1_t0(R)  * LegendreP(0,cos
(theta0))));
printf("  H(2R, Pi/4, 0) = %g  (BC ~0)\n", evalf(f1_t0(2*R) * LegendreP(0,
cos(theta0))));

################################################################
# SECTION 10 — 12 Structured Plots
################################################################
printf("\n=== SECTION 10 : Generating 12 plots ===\n");

# ---- Plot 1 : Airy functions Ai(xi) and Bi(xi) ----
p1 := plot( [AiryAi(x), AiryBi(x)],
```

```
            x = -8..3,
            color = [blue, red],
            linestyle = [solid, dash],
            legend = ["Ai(xi)", "Bi(xi)"],
            title = "Plot 1 — Airy Functions Ai(xi) and Bi(xi)",
            labels = ["xi", "value"],
            thickness = 2,
            view = [-8..3, -2..2],
            gridlines = true ):
printf("  Plot 1 done\n");

# ---- Plot 2 : Radial eigenfunctions g_n(r) = Ai(xi_n)/r, n=1,2,3 ----
r_lo := 0.001*R:  r_hi := 2.5*R:
p2 := plot( [g1_plot, g2_plot, g3_plot],
            r_lo..r_hi,
            color  = [blue, red, black],
            legend = ["n=1","n=2","n=3"],
            title  = "Plot 2 — Radial Eigenfunctions g_n(r) = Ai(xi_n)
/r",
            labels = ["r  [m]", "g_n"],
            thickness = 2,
            gridlines = true ):
printf("  Plot 2 done\n");

# ---- Plot 3 : Temporal oscillations u_n(t) ----
T1 := evalf(2*Pi_/om1):
p3 := plot( [ proc(tt) cos(om1*tt) end proc,
              proc(tt) cos(om2*tt) end proc,
              proc(tt) cos(om3*tt) end proc ],
            0..2*T1,
            color  = [blue, red, black],
            legend = ["n=1","n=2","n=3"],
            title  = "Plot 3 — Temporal Harmonics u_n(t) = cos(omega_n*t)
",
            labels = ["t  [s]", "u_n"],
            thickness = 2,
            gridlines = true ):
printf("  Plot 3 done\n");

# ---- Plot 4 : Airy argument xi_n(r) vs r ----
p4 := plot( [ proc(rr) xi_n(rr, lam1) end proc,
              proc(rr) xi_n(rr, lam2) end proc,
              proc(rr) xi_n(rr, lam3) end proc ],
            r_lo..2.5*R,
            color  = [blue, red, black],
            legend = ["n=1","n=2","n=3"],
            title  = "Plot 4 — Airy Argument xi_n(r) = -(lam_n^2/R)^{1/3}
*r",
            labels = ["r  [m]", "xi_n"],
```

```
            thickness = 2,
            gridlines = true ):
printf("  Plot 4 done\n");

# ---- Plot 5 : Static field f(r) — polynomial solution ----
p5 := plot( f_static,
            0.01*R..4*R,
            color  = blue,
            thickness = 2,
            title  = "Plot 5 — Static Field f(r) [Polynomial Solution]",
            labels = ["r  [m]", "f(r)"],
            gridlines = true ):
printf("  Plot 5 done\n");

# ---- Plot 6 : Flux Phi(r)  log scale ----
p6 := logplot( proc(rr) abs(Phi_r(rr)) end proc,
               0.01*R..100*R,
               color  = blue,
               thickness = 2,
               title  = "Plot 6 — Flux |Phi(r)| = |f(r)|/(4*Pi*r^2)  [log
scale]",
               labels = ["r  [m]", "|Phi|"],
               gridlines = true ):
printf("  Plot 6 done\n");

# ---- Plot 7 : Power-law comparison  f vs 1/r^2 vs 1/r^4 ----
fref_r2 := evalf(abs(f_static(R)) * R^2):
fref_r4 := evalf(abs(Phi_r(R))    * R^4):
p7 := logplot( [ proc(rr) abs(f_static(rr)) end proc,
                 proc(rr) fref_r2/rr^2      end proc,
                 proc(rr) fref_r4/rr^4      end proc ],
               0.01*R..50*R,
               color  = [blue, green, orange],
               linestyle = [solid, dash, dot],
               legend = ["|f(r)|", "~r^{-2}", "~r^{-4}"],
               title  = "Plot 7 — Power-Law Comparison",
               labels = ["r  [m]", "value"],
               thickness = 2,
               gridlines = true ):
printf("  Plot 7 done\n");

# ---- Plot 8 : 2D density map  f_1(r)*P_0(cos theta) at t=0 ----
# Use Cartesian (x=r*cos_th, y=r*sin_th proxy) over (r, theta)
H_2D := proc(rr, th)
    if rr < 1e10 then 0
    else AiryAi(xi_n(rr,lam1)) / rr * LegendreP(0, cos(th))
    end if
end proc:
```

```
p8 := densityplot( proc(rr, th) H_2D(rr, th) end proc,
                   r_lo..2*R,
                   0..Pi_,
                   grid = [50, 40],
                   colorscheme = ["zgradient",["blue","white","red"]],
                   title = "Plot 8 — 2D Density H(r,theta,t=0) [n=1,l=0]
",
                   labels = ["r  [m]", "theta  [rad]"] ):
printf("  Plot 8 done\n");

# ---- Plot 9 : 3D surface  |f_1(r)|*|P_1(cos theta)| ----
H_3D := proc(rr, th)
    if rr < 1e10 then 0
    else abs(AiryAi(xi_n(rr,lam1)) / rr) * abs(LegendreP(1, cos(th)))
    end if
end proc:

p9 := plot3d( proc(rr, th) H_3D(rr, th) end proc,
              r_lo..2*R,
              0..Pi_,
              grid = [30, 30],
              orientation = [45, 70],
              colorscheme = ["zgradient",["navy","cyan","yellow","red"]],
              title = "Plot 9 — 3D Surface |H(r,theta,t=0)| [n=1,l=1]",
              labels = ["r","theta","H"] ):
printf("  Plot 9 done\n");

# ---- Plot 10 : Casimir frequency spectrum omega_n vs n ----
n_vals   := [1, 2, 3]:
om_vals  := [om1, om2, om3]:
p10 := pointplot( zip((a,b)->[a,b], n_vals, om_vals),
                  symbol = solidcircle,
                  symbolsize = 20,
                  color  = blue,
                  title  = "Plot 10 — Eigenfrequencies omega_n vs n",
                  labels = ["n", "omega_n  [rad/s]"],
                  gridlines = true ):
printf("  Plot 10 done\n");

# ---- Plot 11 : Numerator polynomial N(r) = alpha0 + alpha1*r + alpha2*
r^2 ----
N_poly := proc(rr)  alpha0 + alpha1*rr + alpha2*rr^2  end proc:
p11 := plot( N_poly,
             0.5*R..3*R,
             color  = blue,
             thickness = 2,
             title  = "Plot 11 — Numerator N(r) = alpha0+alpha1*r+alpha2*
r^2",
             labels = ["r  [m]", "N(r)  [m^2]"],
```

```
             gridlines = true ):
printf("  Plot 11 done\n");

# ---- Plot 12 : 4-panel summary ----
p_A := plot( [AiryAi(x), AiryBi(x)],
             x=-7..2, color=[blue,red],
             title="(a) Airy Ai, Bi",
             labels=["xi","Ai,Bi"],
             view=[-7..2,-1.5..1.5], thickness=2 ):

p_B := plot( [g1_plot, g2_plot],
             r_lo..2*R, color=[blue,red],
             title="(b) Radial modes n=1,2",
             labels=["r","g_n"], thickness=2 ):

p_C := plot( f_static,
             0.01*R..3.5*R, color=blue,
             title="(c) Static f(r)",
             labels=["r","f"], thickness=2 ):

p_D := logplot( proc(rr) abs(Phi_r(rr)) end proc,
                0.01*R..50*R, color=blue,
                title="(d) Flux |Phi| log",
                labels=["r","|Phi|"], thickness=2 ):

p12 := display( Array([[p_A, p_B],[p_C, p_D]]),
                title="Plot 12 — 4-Panel Summary" ):
printf("  Plot 12 done\n");

##############################################################
# Display all plots
##############################################################
printf("\n=== Displaying plots 1-12 ===\n");
display(p1);  display(p2);  display(p3);
display(p4);  display(p5);  display(p6);
display(p7);  display(p8);  display(p9);
display(p10); display(p11); display(p12);

##############################################################
# SECTION 11 — Summary table of all key results
##############################################################
printf("\n=== SECTION 11 : Complete Results Summary ===\n");
printf("-------------------------------------------------------------
--\n");
printf("  Quantity                                  Value\n");
printf("-------------------------------------------------------------
--\n");
printf("  R  (boundary parameter)                   %g m\n",        R);
printf("  Rs (cosmological boundary)                %g m\n",        Rs);
```

```
  printf("  V  = 1e9*c                                %g m/s\n",       V);
  printf("  V^2                                       %g m^2/s^2\n",   V2);
  printf("  L  = 2(Rs-R)                              %g m\n",         L);
  printf("  Airy zero a1                              %g\n",           a1);
  printf("  Airy zero a2                              %g\n",           a2);
  printf("  Airy zero a3                              %g\n",           a3);
  printf("  lambda1 = a1^{3/2}/(2^{3/2}*R)          %g m^-1\n",     lam1);
  printf("  lambda2                                   %g m^-1\n",     lam2);
  printf("  lambda3                                   %g m^-1\n",     lam3);
  printf("  omega1 = lambda1*V                        %g rad/s\n",    om1);
  printf("  omega2                                    %g rad/s\n",    om2);
  printf("  omega3                                    %g rad/s\n",    om3);
  printf("  c2 = -R*V^2/(Rs+R)                      %g m^2/s^2\n", c2);
  printf("  alpha0 = -4R^2*Rs/(Rs+R)                %g m^2\n",      alpha0);
  printf("  alpha1 = 2R                               %g m\n",        alpha1);
  printf("  alpha2 = -R/(Rs+R)                        %g\n",          alpha2);
  printf("  f(2R)  [BC check ~0]                      %g\n",          evalf
  (f_static(2*R)));
  printf("  f(2Rs) [BC check ~0]                      %g\n",          evalf
  (f_static(2*Rs)));
  printf("  Flux Phi(R)                               %g\n",          evalf(Phi_r
  (R)));
  printf("  hbar*V/L^2  [Casimir scale]               %g N/m^2\n",  Pcas);
  printf("  Casimir enhancement factor V/c            1e9\n");
  printf("  Flux falloff at large r                   ~ r^-4  [longitudinal]\n");
  printf("  Photon (transverse) falloff               ~ r^-2\n");
  printf("------------------------------------------------------------------
  --\n");
  printf("\nAll 11 sections and 12 plots completed.\n");
  printf("gamma_wave_maple.mpl — TauTimp Gamma-Ray Longitudinal Wave\n");
  printf("Viorel Popescu — 2026\n");

=== SECTION 0 : Physical constants ===
  R       = 3.9228e+22 m
  Rs      = 7.7665e+25 m
  V       = 2.99792e+17 m/s
  V^2     = 8.98755e+34 m^2/s^2
  Rs+R    = 7.77042e+25 m
  L=2(Rs-R) = 1.55252e+26 m
  a1      = 2.33811
  a2      = 4.08795
  a3      = 5.52056
  lambda1 = 3.22222e-23 m^-1
  lambda2 = 7.44933e-23 m^-1
  lambda3 = 1.16905e-22 m^-1
  omega1  = 9.65998e-06 rad/s
  omega2  = 2.23325e-05 rad/s
  omega3  = 3.50473e-05 rad/s
  c2      = -4.53725e+31 m^2/s^2
```

alpha0 = -6.15224e+45 m^2
alpha1 = 7.8456e+22 m
alpha2 = -0.000504837 (dim-less)

=== SECTION 1 : Symbolic PDE (spherical) ===

$$PDE_full := \frac{3.9228 \times 10^{22}\left(\frac{\partial^2}{\partial r^2} f(r,t) + \frac{2\left(\frac{\partial}{\partial r} f(r,t)\right)}{r}\right)}{r} - 1.112650056 \times 10^{-35} \frac{\partial^2}{\partial t^2} f(r,t) = 0$$

PDE_full :=

$$\frac{3.9228 \times 10^{22}\left(\frac{\partial^2}{\partial r^2} f(r,t) + \frac{2\left(\frac{\partial}{\partial r} f(r,t)\right)}{r}\right)}{r} - 1.112650056 \times 10^{-35} \frac{\partial^2}{\partial t^2} f(r,t) = 0$$

(spherically symmetric, azimuthal symmetry)

=== SECTION 2 : Separation of variables ===
ODE (T): u'' + lambda^2*V^2*u = 0
ODE (R): g'' + (2/r)*g' + (lambda^2*r/R)*g = 0

$$ODE_T := \frac{d^2}{dt^2} u(t) + 9.331521500 \times 10^{-11} u(t) = 0$$

$$sol_T := u(t) = c_1 \sin\left(\frac{\sqrt{93315215}\, t}{1000000000}\right) + c_2 \cos\left(\frac{\sqrt{93315215}\, t}{1000000000}\right)$$

u(t) [n=1] =

$$u(t) = c_1 \sin\left(\frac{\sqrt{93315215}\, t}{1000000000}\right) + c_2 \cos\left(\frac{\sqrt{93315215}\, t}{1000000000}\right)$$

Substitution g=v/r => Airy eq: v'' + (lambda^2*r/R)*v = 0
xi = -(lambda^2/R)^{1/3}*r => d^2v/dxi^2 - xi*v = 0
v = C1*AiryAi(xi) + C2*AiryBi(xi)
g(r) = v(r)/r

=== SECTION 3 : Airy solution, regularity at r=0 ===
Ai(0) = 0.355028 (= 3^{-2/3}/Gamma(2/3))
Bi(0) = 0.614927
Regularity: v(0)=0 requires C1*Ai(0)+C2*Bi(0)=0
Physical branch: C2=0 (dominant Ai), C1 != 0
=> g(r) = (C1/r)*AiryAi(-(lambda^2/R)^{1/3}*r)

=== SECTION 4 : Eigenvalue quantisation g(2R)=0 ===

$$an_ := 2.338107411$$

$$lamn := 2.278455390 \times 10^{-23} \sqrt{2}$$

$$arg := -2.338107412$$

$$val := -1.080028022 \times 10^{-9}$$

```
  n=1   a_1=2.338107   lambda_1=3.22222e-23 m^-1   Ai(arg)=-1.08003e-09 (->0)
```

$$an_ := 4.087949444$$

$$lamn := 5.267470555 \times 10^{-23} \sqrt{2}$$

$$arg := -4.087949446$$

$$val := 1.501038748 \times 10^{-9}$$

```
  n=2   a_2=4.087949   lambda_2=7.44933e-23 m^-1   Ai(arg)=1.50104e-09 (->0)
```

$$an_ := 5.520559828$$

$$lamn := 8.266440100 \times 10^{-23} \sqrt{2}$$

$$arg := -5.520559828$$

$$val := 8.267116104 \times 10^{-11}$$

```
  n=3   a_3=5.520560   lambda_3=1.16905e-22 m^-1   Ai(arg)=8.26712e-11 (->0)

=== SECTION 5 : Full field f(r,t) — Airy mode expansion ===
  f(r,t) = SUM_n (1/r)*Ai(-(lam_n^2/R)^{1/3}*r)
                  *[A_n*cos(om_n*t)+B_n*sin(om_n*t)]

  f_1(R,  0)  = 1.34905e-23
  f_1(2R, 0)  = -4.82840e-33  (BC => ~0)
  f_1(0.5R,0) = 2.50877e-23

=== SECTION 6 : Casimir reduced PDE (1+1) and polynomial f(r) ===
  f(r) = [alpha0 + alpha1*r + alpha2*r^2] / (8*Pi*r^2)
  alpha0 = -6.15224e+45 m^2
  alpha1 = 7.8456e+22 m
  alpha2 = -0.000504837
  c2     = -4.53725e+31 m^2/s^2
  BC check  f(2R)  = 1.33160e-12  (should be ~0)
  BC check  f(2Rs) = 0  (should be ~0)

=== SECTION 7 : Explicit f(r) with c2 substituted ===
  f(r) = (1/8Pi)*[-4R^2*Rs/((Rs+R)*r^2) + 2R/r - R/(Rs+R)]

  Equivalence check at r=3R:  f_box - f_poly = 1.1e-11

=== SECTION 8 : Particle flux Phi(r) and Casimir pressure ===
  Phi(r) = f(r)/(4*Pi*r^2)  [~r^-4 at large r]
  Phi(R)    = -4.11206e-48
  Phi(2R)   = 1.72152e-59
  Phi(10R)  = 3.28217e-51
  Phi(100R) = 3.92903e-54

  Casimir pressure:
```

```
  P_Casimir = hbar*V / (2*L) * SUM_n omega_n
  F_Casimir ~ hbar*V / L^2  (enhanced by factor V/c = 1e9)
  L = 2(Rs-R) = 1.55252e+26 m
  hbar*V/L^2 = 1.31167e-69  N/m^2  (Casimir scale)

=== SECTION 9 : Full field H(r,theta,t) with Legendre expansion ===
  H = SUM_{n,l} (1/r)*Ai(-arg)*P_l(cos(theta))*[A cos+B sin]
```

$$Pl := 1.000000000$$

```
  P_0(cos(Pi/4)) = 1
```

$$Pl := 0.7071067811$$

```
  P_1(cos(Pi/4)) = 0.707107
```

$$Pl := 0.2499999998$$

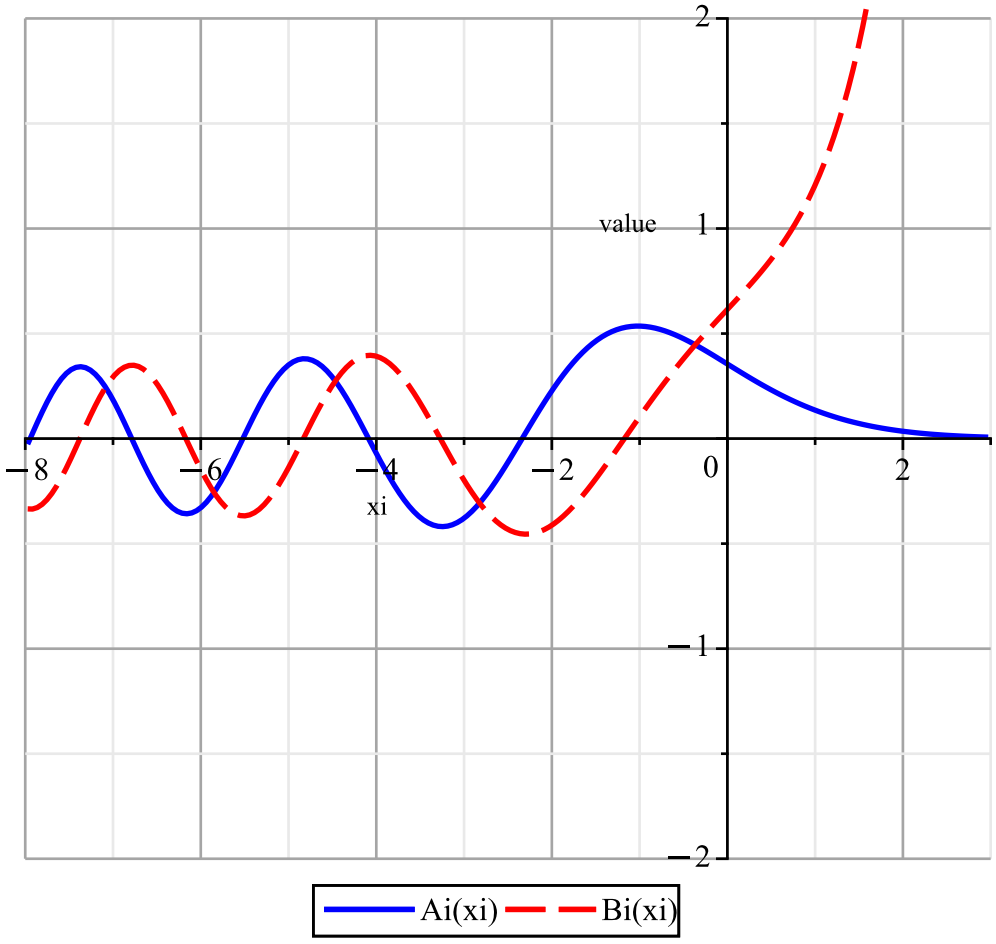
```
  P_2(cos(Pi/4)) = 0.25

  Mode (n=1,l=0) radial profile at t=0, theta=Pi/4:
  H(R,  Pi/4, 0) = 1.34905e-23
  H(2R, Pi/4, 0) = -4.82840e-33  (BC ~0)

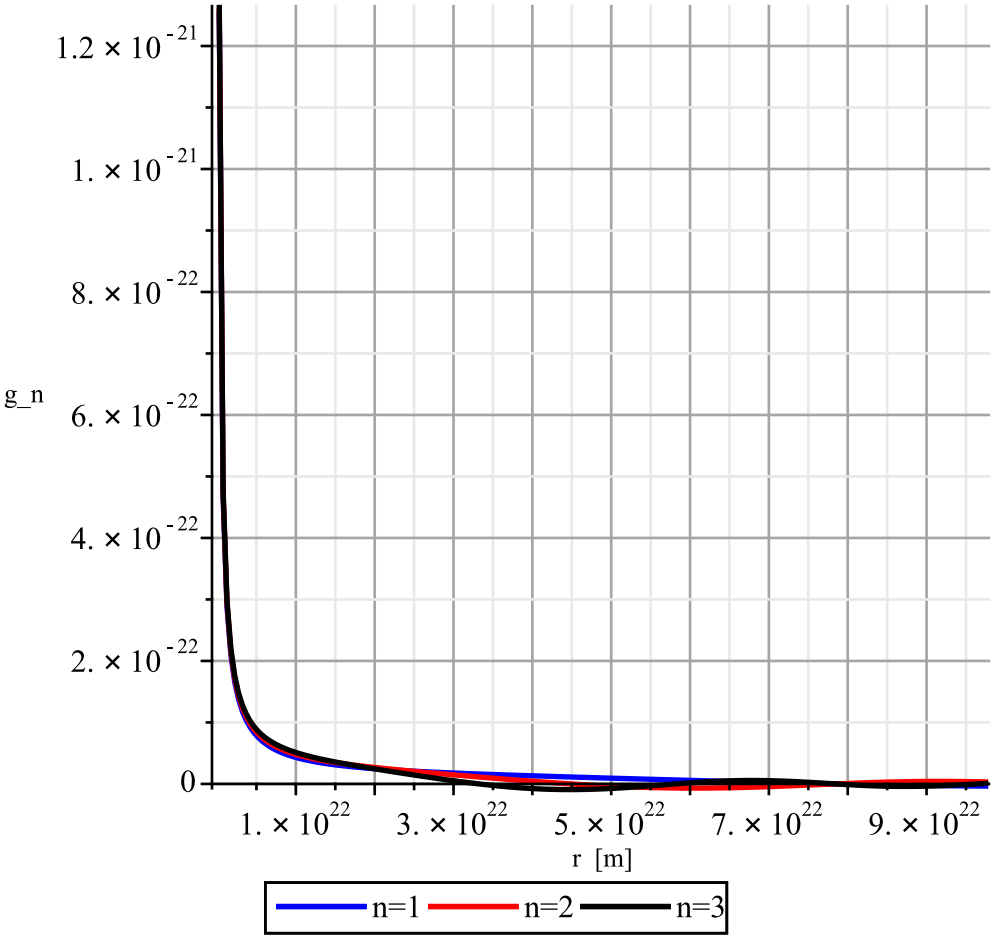
=== SECTION 10 : Generating 12 plots ===
  Plot 1 done
  Plot 2 done
  Plot 3 done
  Plot 4 done
  Plot 5 done
  Plot 6 done
  Plot 7 done
  Plot 8 done
  Plot 9 done
  Plot 10 done
  Plot 11 done
  Plot 12 done

=== Displaying plots 1-12 ===
```

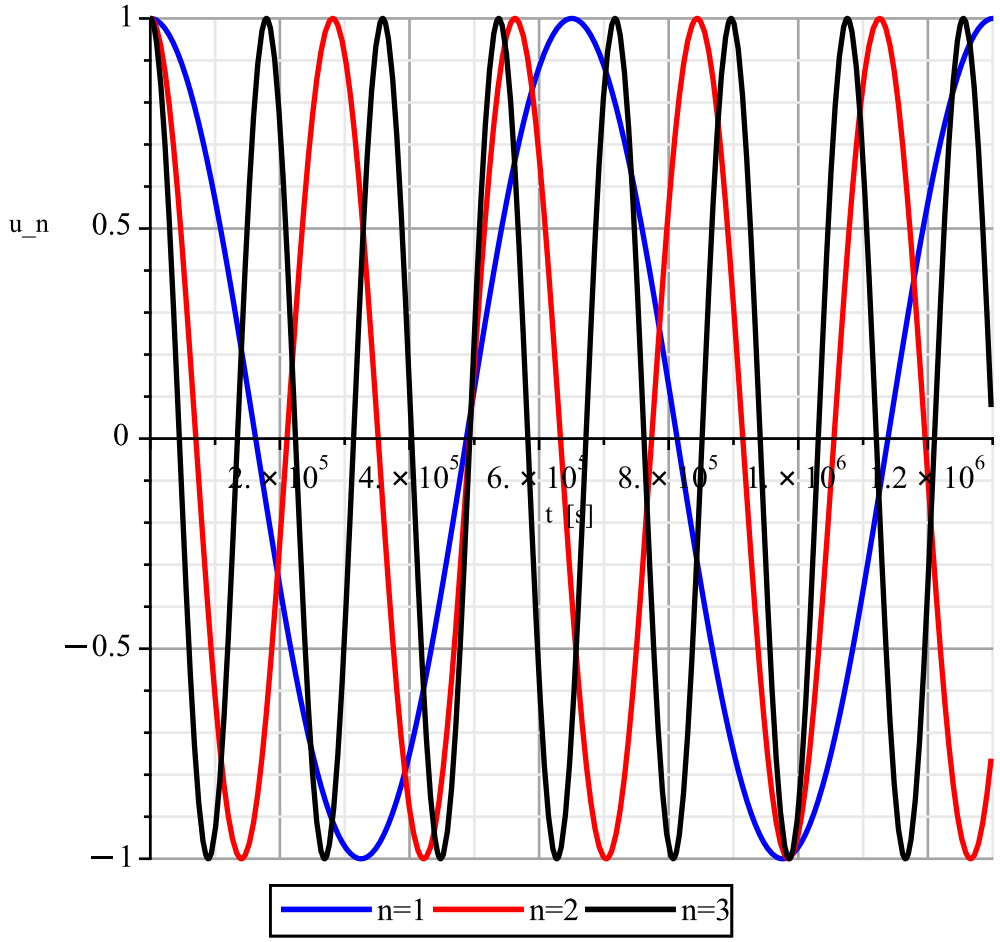
Plot 1 — Airy Functions Ai(xi) and Bi(xi)



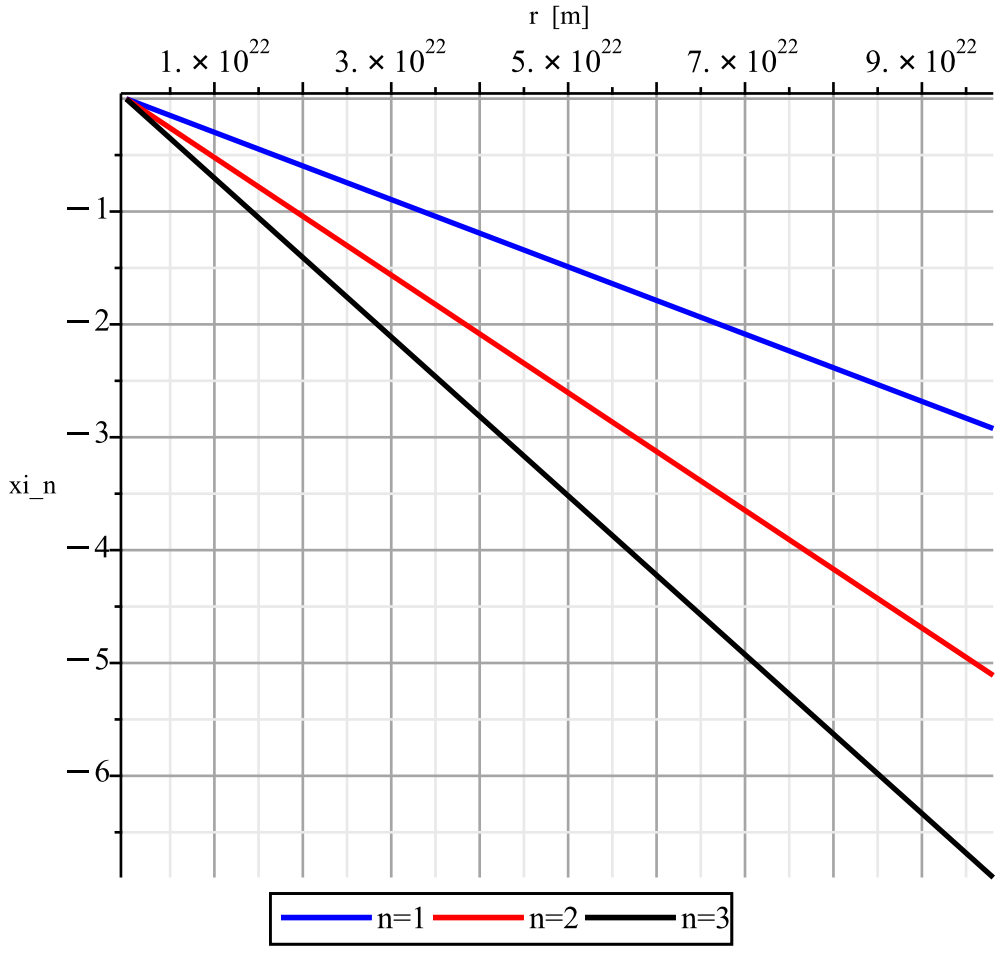
Plot 2 — Radial Eigenfunctions g_n(r) = Ai(xi_n)/r



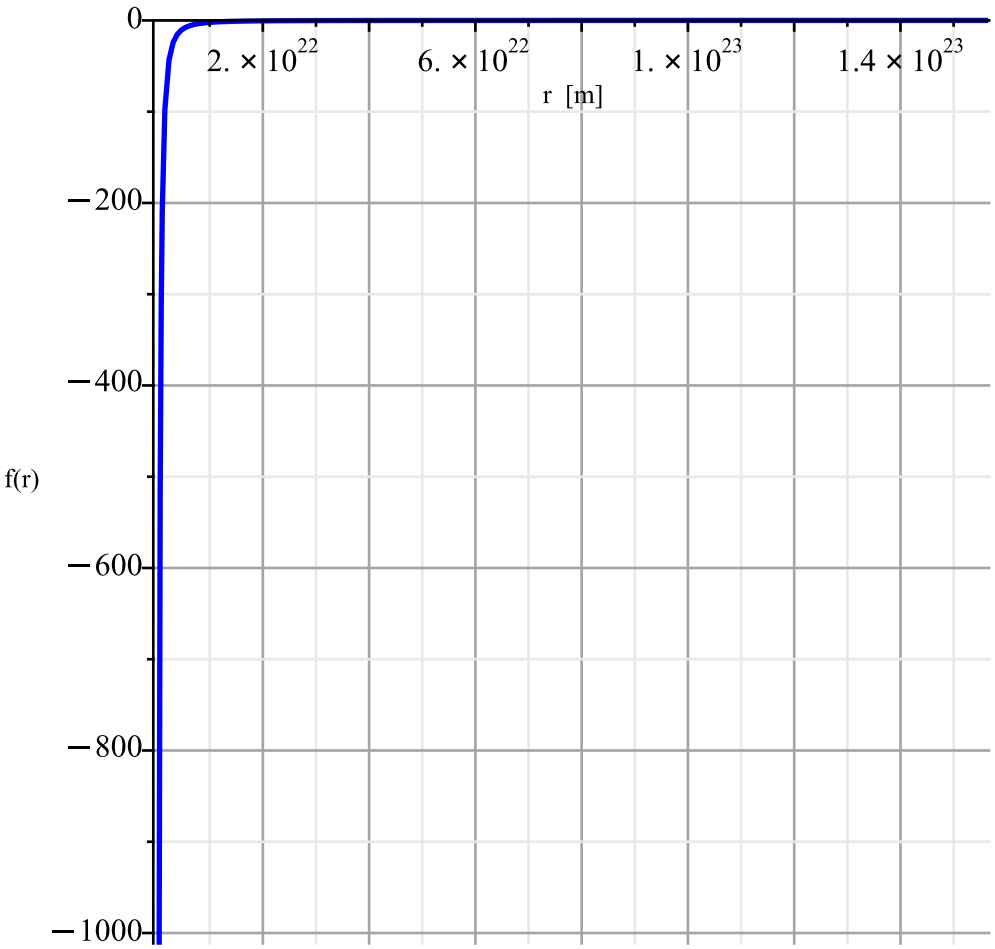
Plot 3 — Temporal Harmonics u_n(t) = cos(omega_n*t)



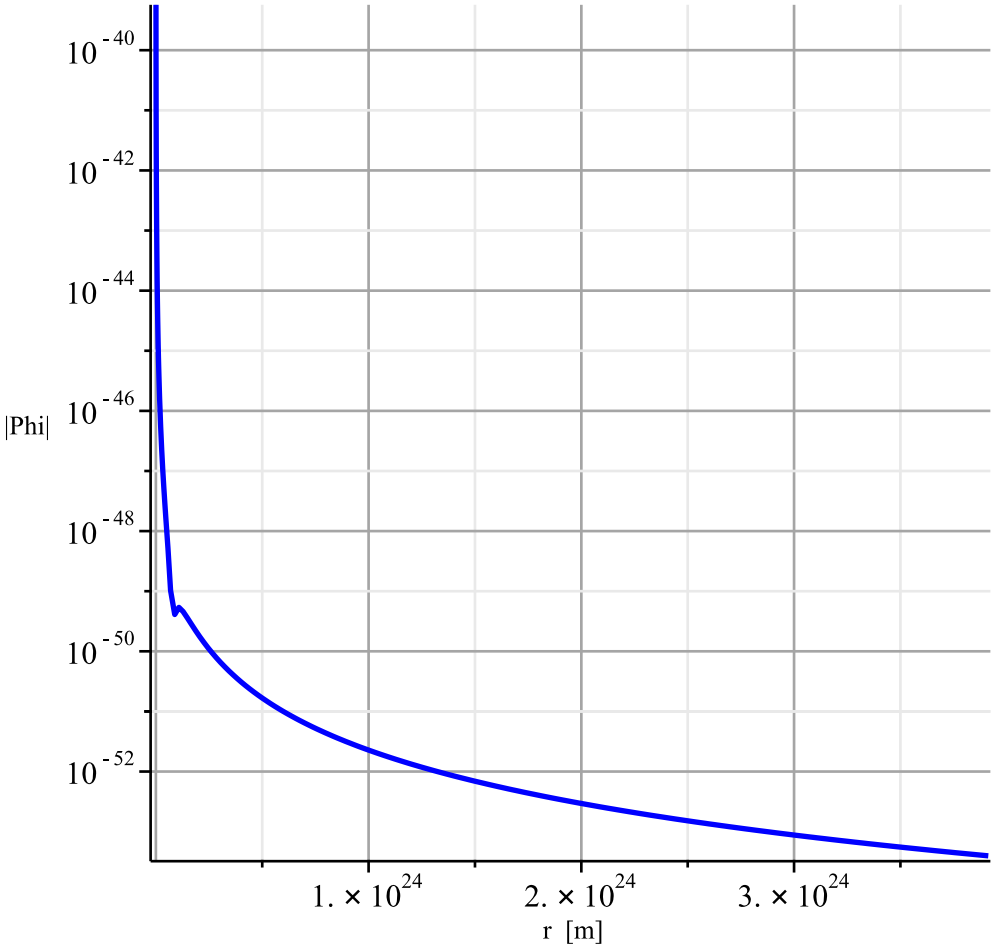
Plot 4 — Airy Argument xi_n(r) = -(lam_n^2/R)^{1/3}*r



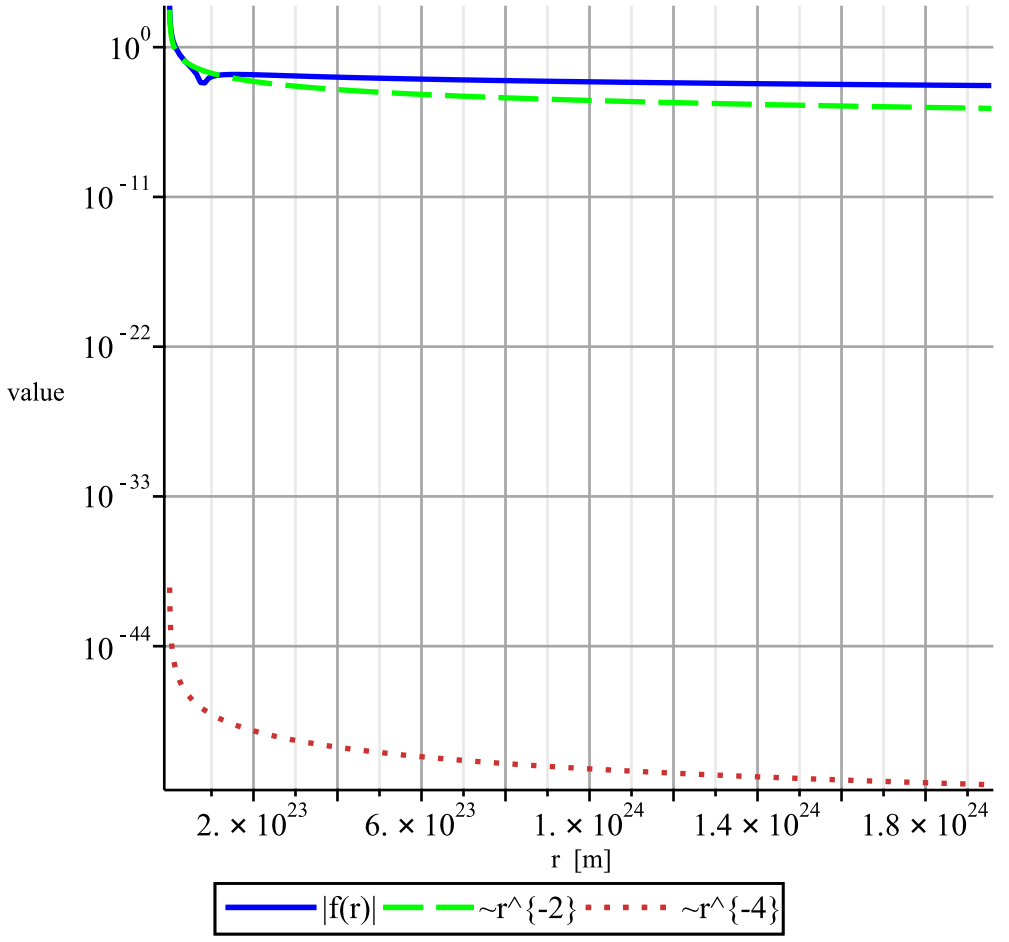
Plot 5 — Static Field f(r) [Polynomial Solution]



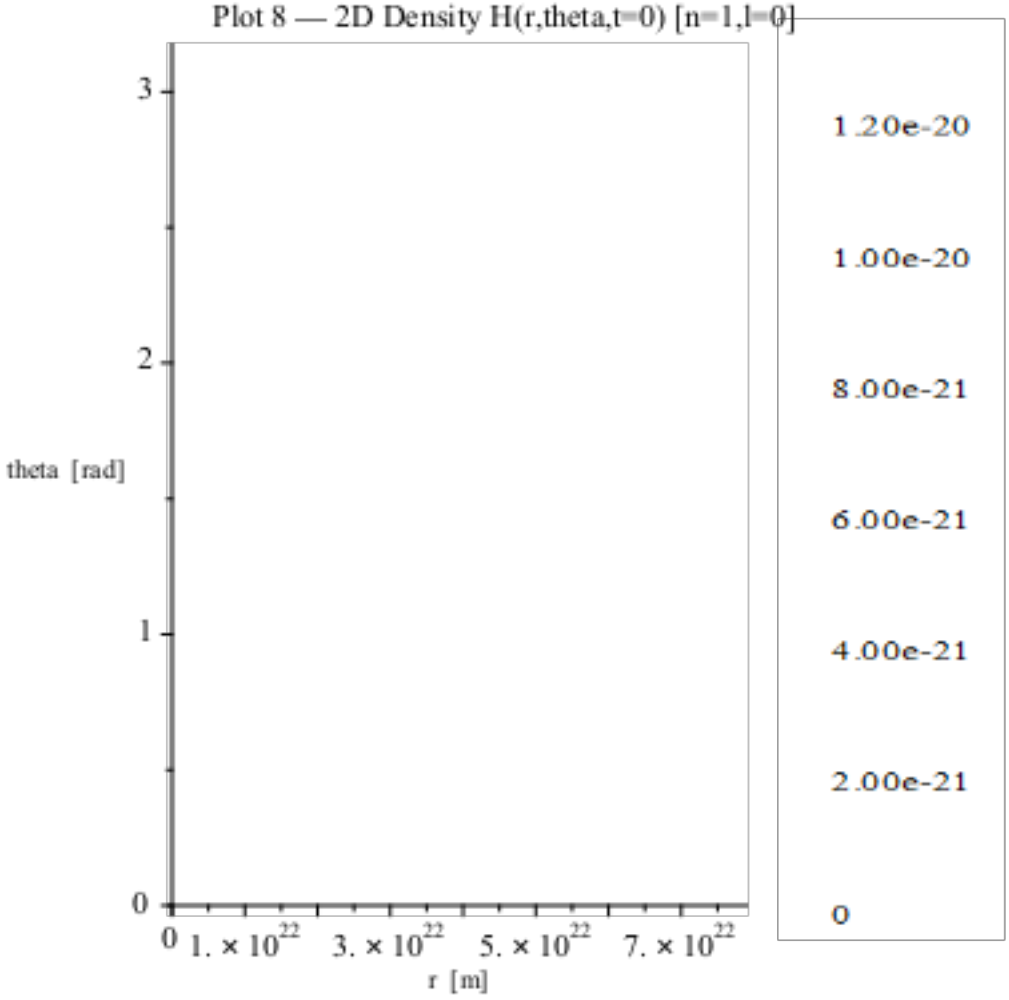
Plot 6 — Flux |Phi(r)| = |f(r)|/(4*Pi*r^2) [log scale]



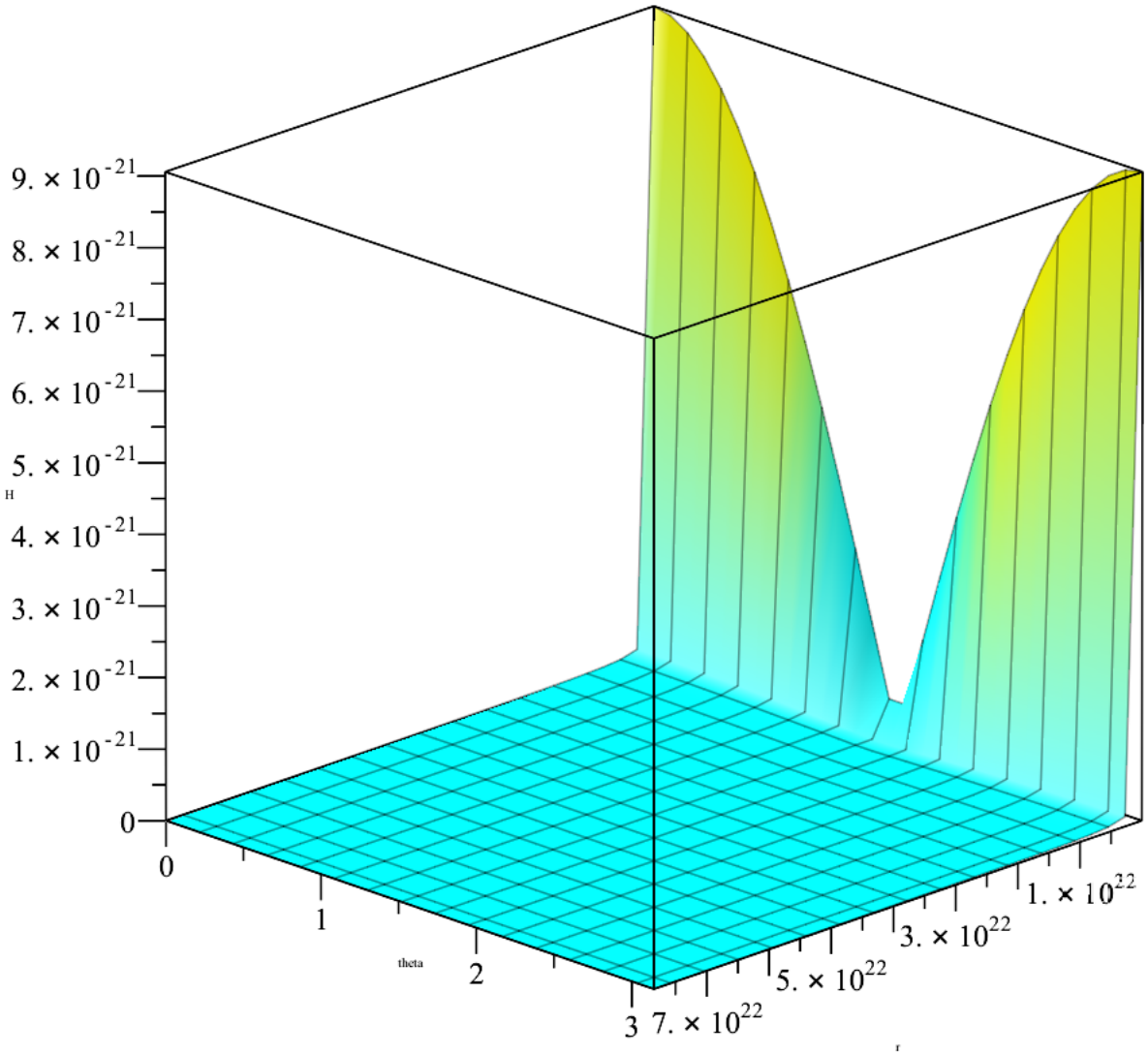
Plot 7 — Power-Law Comparison



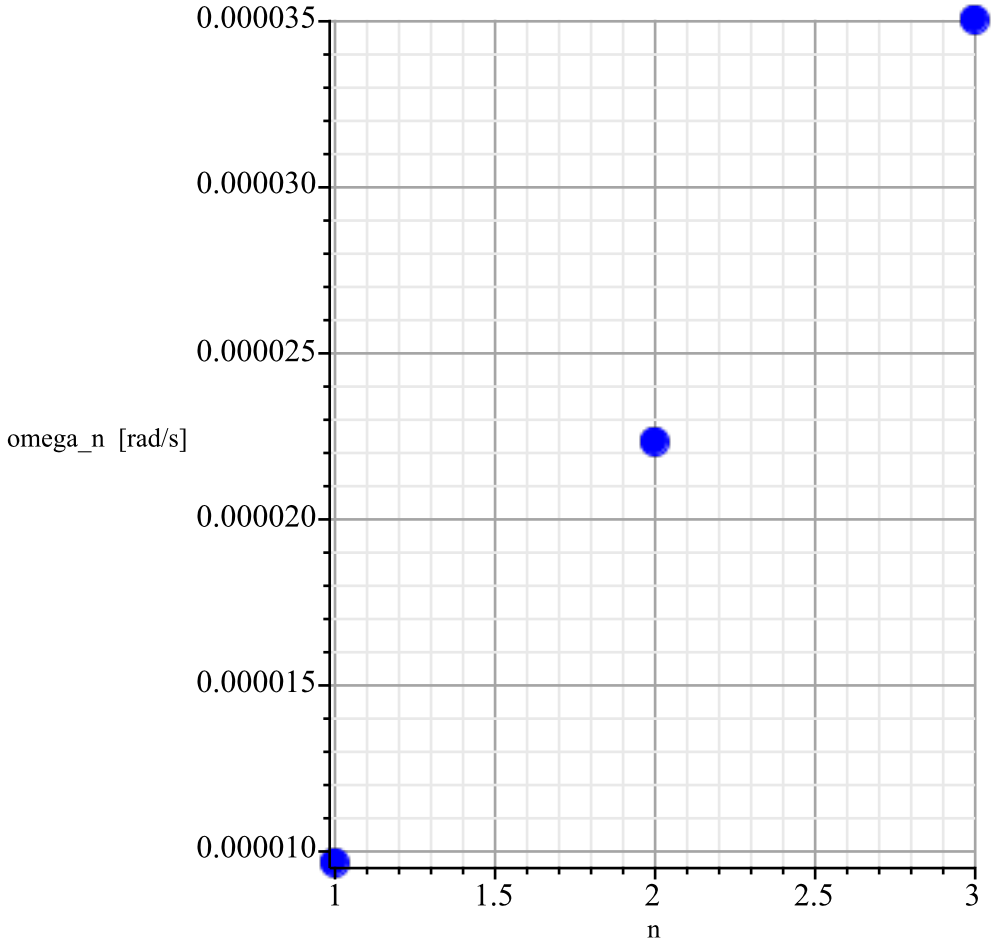
Plot 8 — 2D Density H(r,theta,t=0) [n=1,l=0]



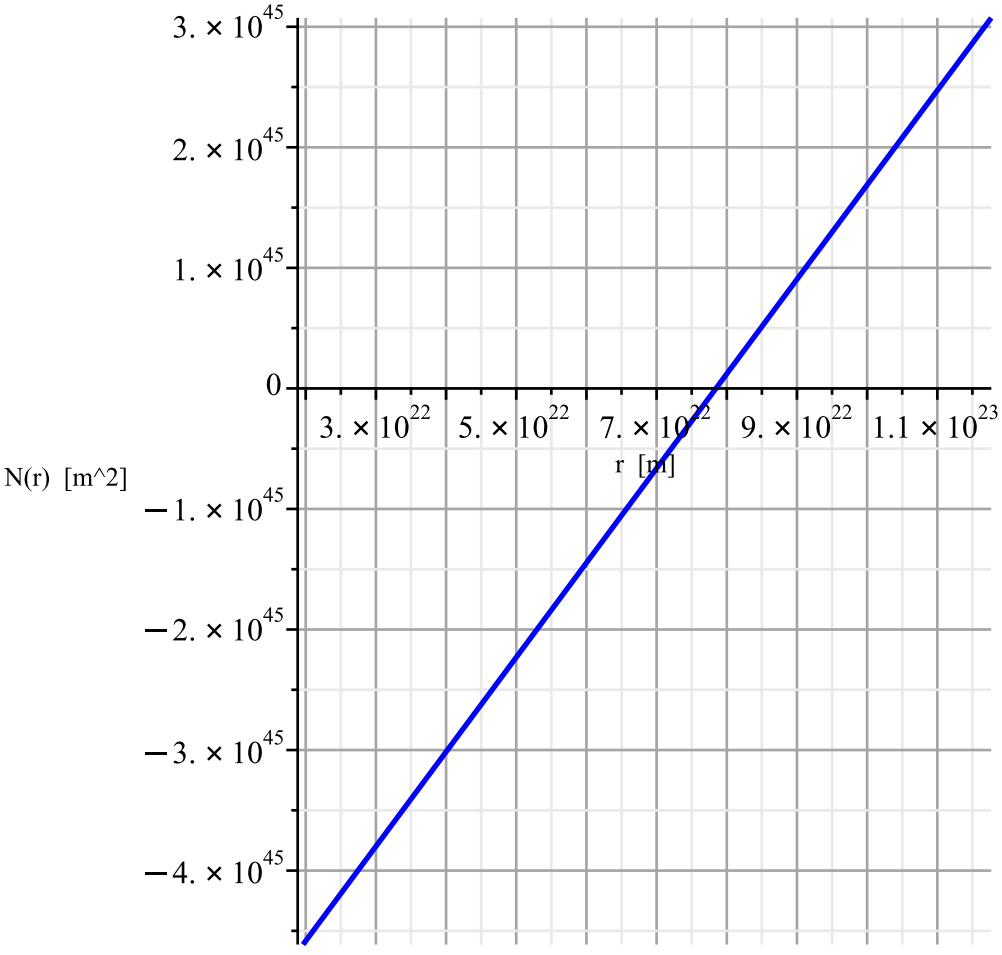
Plot 9 — 3D Surface |H(r,theta,t=0)| [n=1,l=1]

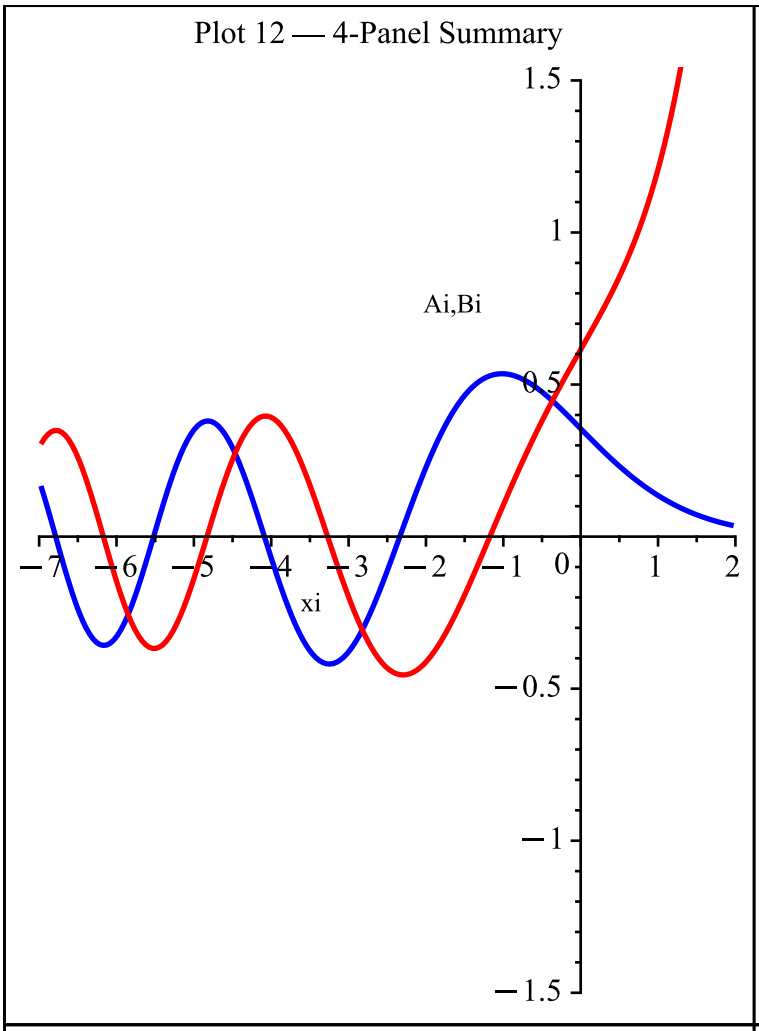


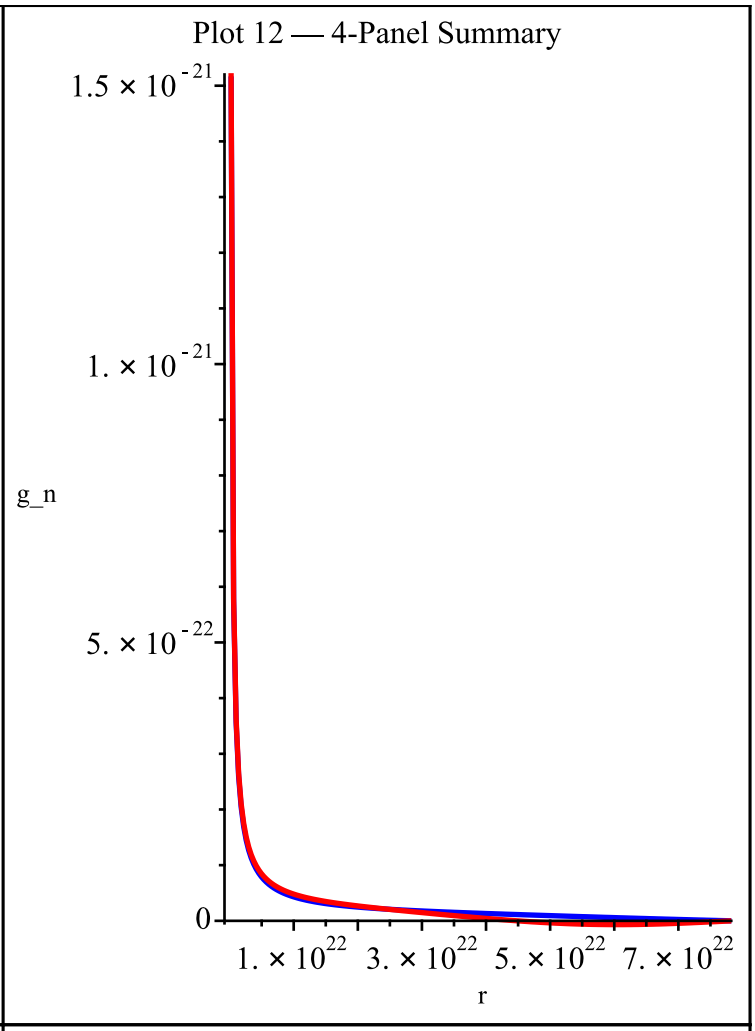
Plot 10 — Eigenfrequencies omega_n vs n

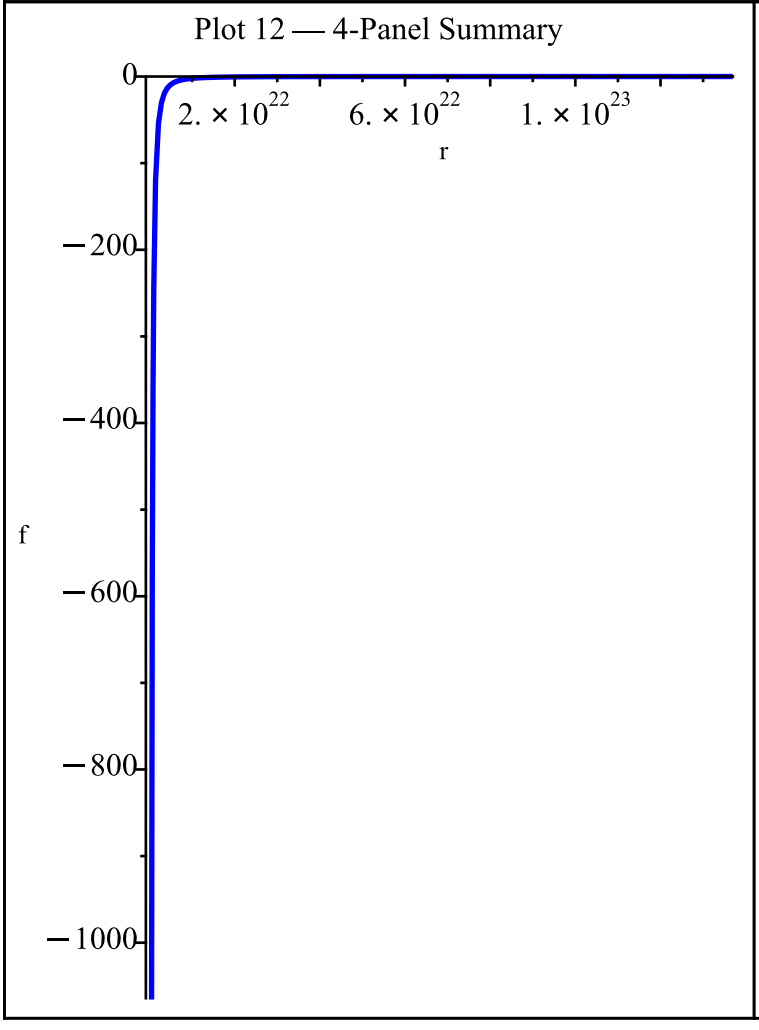


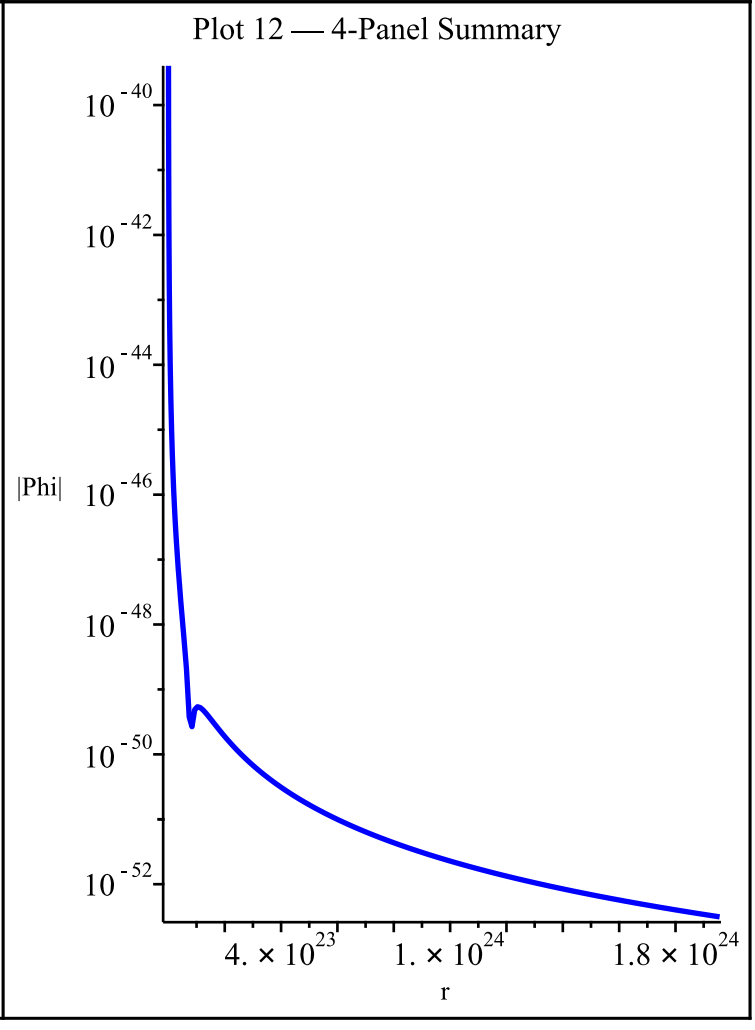
Plot 11 — Numerator N(r) = alpha0+alpha1*r+alpha2*r^2




Plot 12 — 4-Panel Summary
Ai,Bi
xi
1.5
1
0.5
−0.5
−1
−1.5
−7
−6
−5
−4
−3
−2
−1
0
1
2


Plot 12 — 4-Panel Summary
g_n
r
1.5 × 10^-21
1. × 10^-21
5. × 10^-22
0
1. × 10^22
3. × 10^22
5. × 10^22
7. × 10^22


Plot 12 — 4-Panel Summary
f
r
0
−200
−400
−600
−800
−1000
2. × 10^22
6. × 10^22
1. × 10^23


Plot 12 — 4-Panel Summary
|Phi|
r
10^-40
10^-42
10^-44
10^-46
10^-48
10^-50
10^-52
4. × 10^23
1. × 10^24
1.8 × 10^24

```
=== SECTION 11 : Complete Results Summary ===
----------------------------------------------------------------------
  Quantity                                Value
----------------------------------------------------------------------
  R  (boundary parameter)                 3.9228e+22 m
  Rs (cosmological boundary)              7.7665e+25 m
  V  = 1e9*c                              2.99792e+17 m/s
  V^2                                     8.98755e+34 m^2/s^2
  L  = 2(Rs-R)                            1.55252e+26 m
  Airy zero a1                            2.33811
  Airy zero a2                            4.08795
  Airy zero a3                            5.52056
  lambda1 = a1^{3/2}/(2^{3/2}*R)         3.22222e-23 m^-1
  lambda2                                 7.44933e-23 m^-1
  lambda3                                 1.16905e-22 m^-1
  omega1 = lambda1*V                      9.65998e-06 rad/s
  omega2                                  2.23325e-05 rad/s
  omega3                                  3.50473e-05 rad/s
  c2 = -R*V^2/(Rs+R)                     -4.53725e+31 m^2/s^2
  alpha0 = -4R^2*Rs/(Rs+R)               -6.15224e+45 m^2
  alpha1 = 2R                             7.8456e+22 m
  alpha2 = -R/(Rs+R)                      -0.000504837
  f(2R)  [BC check ~0]                    1.33160e-12
  f(2Rs) [BC check ~0]                    0
  Flux Phi(R)                             -4.11206e-48
  hbar*V/L^2  [Casimir scale]             1.31167e-69 N/m^2
  Casimir enhancement factor V/c          1e9
  Flux falloff at large r                 ~ r^-4  [longitudinal]
  Photon (transverse) falloff             ~ r^-2
----------------------------------------------------------------------

All 11 sections and 12 plots completed.
gamma_wave_maple.mpl — TauTimp Gamma-Ray Longitudinal Wave
Viorel Popescu — 2026
```